

PASSWORD CRACKING & CREDENTIAL ATTACK SUITE

Project Report

Prepared By: Amandeep Singh
Project Domain: Cybersecurity / SOC Analyst
Technology: Python 3
Testing Environment: Authorized Laboratory Environment

Ethical Disclaimer: This project is designed for educational and authorized security testing only. It uses self-created laboratory test data and does not target real accounts, systems, or credentials.

1. Abstract

Password security is an important part of cybersecurity because weak and predictable passwords can increase the risk of unauthorized access, credential stuffing, brute-force attacks, and data breaches.

The Password Cracking & Credential Attack Suite is a Python-based educational toolkit that demonstrates password security auditing in a controlled environment.

The project analyzes password strength, estimates entropy, generates a custom test dictionary, creates and verifies SHA-256 hashes, performs a limited dictionary attack simulation, performs a controlled brute-force simulation, and produces a security audit report.

The project helps learners understand both offensive password-guessing concepts and defensive authentication controls.

2. Introduction

Passwords are commonly used as the first layer of authentication. When users choose short, common, or predictable passwords, attackers may have a greater chance of guessing them.

Security teams therefore need methods to evaluate password policies and understand the risks associated with weak credentials.

This project provides a simple practical demonstration of these concepts. Instead of attacking real accounts, the application creates and tests passwords and hashes locally.

This makes the project suitable for cybersecurity learning, demonstrations, and portfolio development.

3. Problem Statement

Organizations need to identify weak authentication practices before attackers can exploit them.

A security analyst should understand:

- I.
- II. How password guessing works
- III. How password hashes are compared
- IV. Why dictionary attacks can be effective
- V. Why predictable passwords are dangerous
- VI. Which defensive controls reduce brute-force risk

This project provides a controlled environment to demonstrate these concepts.

4. Project Objectives

The main objectives are:

- I. Analyze password length, character types, and estimated entropy.
- II. Generate custom password-testing dictionaries.
- III. Generate and verify SHA-256 hashes for laboratory passwords.
- IV. Simulate dictionary-based password testing.
- V. Demonstrate a limited brute-force methodology.
- VI. Identify weak password characteristics.
- VII. Generate a security audit report.
- VIII. Provide recommendations for improving authentication security.
- IX. Understand red-team and blue-team password security concepts.

5. Project Features

5.1 Password Strength Analyzer

The analyzer checks:

Password length
Lowercase characters
Uppercase characters
Numbers
Special characters
Common-password patterns

Estimated entropy

Example:

Password: abc123

Strength: WEAK

5.2 Dictionary Generator

The dictionary generator creates password candidates from sample words.

Example input:

admin

password

security

kali

Possible generated candidates:

admin

admin123

admin1234

admin2026

Admin

Admin123

ADMIN

This demonstrates how predictable password variations can increase security risk.

5.3 Hash Generator and Checker

The project uses Python's hashlib library to create SHA-256 hashes for laboratory test passwords.

The process is:

Test Password

↓

SHA-256

↓

Password Hash

↓

Compare Candidate

↓

Match / No Match

5.4 Dictionary Attack Simulation

The application reads candidates from the generated wordlist and compares them with the laboratory-generated hash.

Example:

Candidate: admin

Match: NO

Candidate: password

Match: NO

Candidate: admin123

Match: YES

5.5 Controlled Brute-Force Simulation

The project contains a deliberately limited brute-force demonstration.

It tests lowercase combinations up to a small configurable length.

Example:

a
b
c
...
aa
ab
ac
...
abc

The limitation keeps the demonstration suitable for an educational lab.

5.6 Security Audit Report

The application generates:

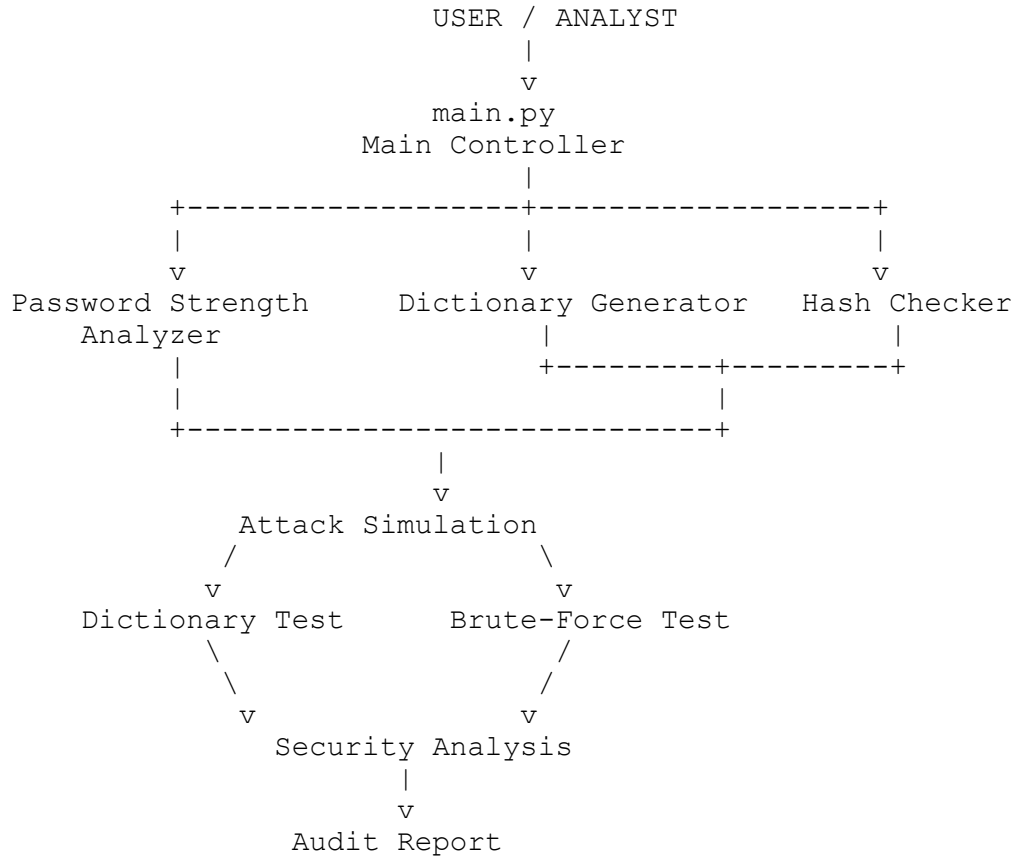
reports/audit_report.txt

The report contains:

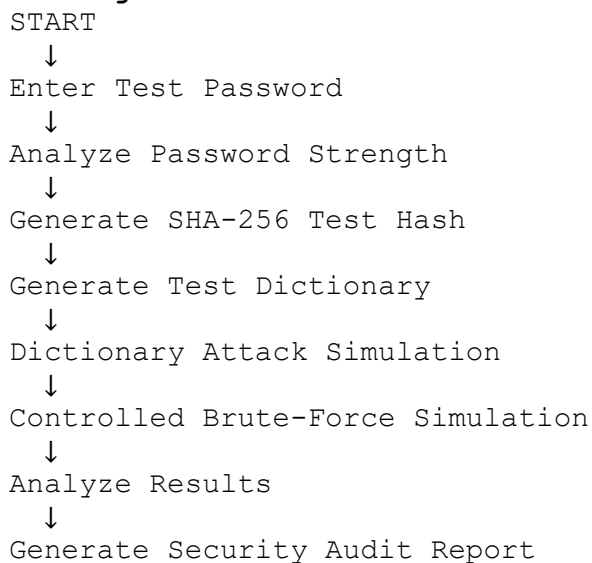
- I.
- II. Password strength
- III. Password characteristics
- IV. Estimated entropy
- V. Dictionary simulation result
- VI. Brute-force simulation result
- VII. Overall risk
- VIII. Security recommendations
- IX. 6. Technologies Used
- X. Technology Purpose
- XI. Python 3 Main programming language
- XII. hashlib SHA-256 hashing

- XIII. `itertools` Brute-force combinations
- XIV. `string` Character handling
- XV. `mathEntropy` calculation
- XVI. `pytest` Automated testing
- XVII. Kali Linux / Ubuntu Laboratory environment
- XVIII.** `Git` / `GitHub` Version control

7. Project Architecture



8. Project Workflow



↓
END
9. Project Structure
Password-Cracking-Credential-Audit/
|
├─ app/
| ├── __init__.py
| ├── password_strength.py
| ├── dictionary_generator.py
| ├── hash_checker.py
| ├── brute_force.py
| └─ report.py
|
├─ data/
| ├── sample_words.txt
| └─ wordlist.txt
|
├─ reports/
| └─ audit_report.txt
|
├─ screenshots/
|
├─ tests/
| └─ test_password_security.py
|
├─ main.py
├─ requirements.txt
├─ README.md
└─ venv/

10. Implementation

The main program accepts a test password and sends it through different modules.

The password strength module calculates complexity information.

The dictionary generator creates candidate passwords.

The hash checker generates a SHA-256 hash and compares candidate passwords with the target hash.

The simulation modules return the number of attempts and whether a matching candidate was found.

Finally, the report module writes the results to an audit file.

11. Password Strength Analysis

The analyzer checks whether the password contains:

Lowercase letters
Uppercase letters
Numbers
Special characters

It also considers password length and common-password patterns.

A simple entropy estimate is calculated based on the approximate character pool used by the password.

12. Dictionary Simulation

The dictionary module creates a small test wordlist from sample words such as:

admin
password
security
kali
test

Variations are generated by changing capitalization or adding predictable number suffixes.

The resulting candidates are compared with the locally generated test hash.

13. Brute-Force Simulation

The brute-force module demonstrates the basic process of generating combinations.

For safety, the implementation is intentionally restricted to:

Character set: lowercase letters
Maximum length: 3

This is enough to demonstrate the concept without creating a practical credential attack tool.

14. Example Test Results

Test Case 1 — Weak Password
Password: admin123

Strength: WEAK
Dictionary Simulation: PASSWORD FOUND
Overall Risk: HIGH

Reason:

Common word
Predictable number suffix
No uppercase characters
No special characters
Relatively short
Test Case 2 – Strong Password
Password: T9#vLm7@Qx2!

Strength: STRONG
Dictionary Simulation: NOT FOUND

The second password is significantly more difficult to guess because it has greater length and character diversity.

15. Automated Testing

The project contains:

tests/test_password_security.py

Run:

pytest

Expected:

tests/test_password_security.py [100%]

4 passed

The tests verify:

- I. Weak password detection
 - II. Strong password detection
 - III. Correct hash matching
 - IV. Incorrect password rejection
16. Security Audit Report

The application generates:

reports/audit_report.txt

It contains:

- I. PASSWORD STRENGTH
- II. DICTIONARY SIMULATION
- III. BRUTE-FORCE SIMULATION
- IV. OVERALL RISK
- V. RECOMMENDATIONS

Example:

OVERALL RISK

Risk Level: HIGH

RECOMMENDATIONS

- 1. Use long and unique passwords.
2. Avoid common dictionary words.
3. Implement multi-factor authentication.
4. Implement login rate limiting.
5. Never reuse passwords.
6. Use secure password hashing mechanisms.
17. Red-Team Concepts

The project demonstrates the concepts of:

- I. Password dictionary attacks
- II. Password mutations
- III. Hash comparison
- IV. Brute-force methodology
- V. Password predictability

These concepts help security professionals understand how weak credentials may be tested during authorized assessments.

18. Blue-Team Concepts

From the defensive perspective, the project highlights:

- I. Strong password policies
- II. Multi-factor authentication
- III. Login rate limiting
- IV. Account lockout controls
- V. Authentication logging
- VI. Monitoring repeated failed login attempts

A SOC analyst could monitor:

Failed Login



Repeated Attempts



Possible Brute Force



SIEM Alert



Investigation



Response

19. Security Recommendations

Organizations should:

- I. Use long and unique passwords.
- II. Avoid predictable password patterns.
- III. Enable multi-factor authentication.
- IV. Implement login rate limiting.
- V. Monitor authentication failures.
- VI. Never store passwords in plaintext.
- VII. Use appropriate password-hashing mechanisms.
- VIII. Use unique salts and suitable work factors.
- IX. Detect repeated failed authentication attempts.
- X. Educate users about password security.

20. Limitations

This project is an educational simulation.

It does not:

- I. Extract real Linux shadow credentials
- II. Extract Windows SAM credentials
- III. Perform online authentication attacks
- IV. Use stolen credentials
- V. Bypass authentication
- VI. Attack external systems
- VII. Target real user accounts

The brute-force demonstration is intentionally limited.

21. Future Enhancements

Future versions could include:

- I. GUI interface using Tkinter
- II. PDF/JSON report generation
- III. Configurable password policies
- IV. Larger password-risk database
- V. Authentication-log analysis
- VI. Wazuh integration
- VII. Splunk integration
- VIII. Brute-force detection rules
- IX. Interactive dashboard
- X. Password-policy compliance scoring

22. Learning Outcomes

After completing this project, I learned:

- I. How password strength can be evaluated
- II. Why predictable passwords are dangerous
- III. How password hashes can be compared
- IV. How dictionary attacks work conceptually
- V. How brute-force attacks work conceptually

- VI. Why password length is important
- VII. Why MFA is important
- VIII. Why authentication rate limiting is important
- IX. How security audit reports are generated
- X. How Python can be used for cybersecurity automation
- XI. How red-team and blue-team concepts are connected

23. Conclusion

The Password Cracking & Credential Attack Suite demonstrates why password security is important and how weak passwords can be identified through controlled testing.

The project combines offensive security concepts such as dictionary attacks and brute-force methodology with defensive security recommendations such as strong password policies, MFA, rate limiting, and authentication monitoring.

This project provides practical experience in Python, cybersecurity automation, password security, hashing, security auditing, testing, and SOC-related defensive concepts.

All testing should remain within an authorized laboratory environment.